DJANGO MASTER CURRICULUM

2026 Backend Engineer Roadmap

6 Months • 2 Hours Daily • Saturday Reviews • 2 Deployed Projects

Technology Stack — What You Learn and When

PHASE	TECHNOLOGY INTRODUCED
Week 0	Python 3.11, Git, SQLite, Django 4.2 basics
Phase 1 — Month 1	Django 4.2, Docker, Postgres 15, WSL2, python-decouple
Phase 2 — Month 2	DRF, SimpleJWT, django-filter, drf-spectacular, Postman
Phase 3 — Month 3	Redis, Celery, celery-beat, django-celery-beat, MinIO, Flower, Pillow
Phase 4 — Month 4	Scrapy / BeautifulSoup, LangChain, OpenAI API, Ollama (optional)
Phase 4 — Month 5	Django Channels, Daphne, WebSockets, ASGI
Phase 4 — Month 6	Pytest, pytest-cov, GitHub Actions, Render, unittest.mock

Daily Non-Negotiable Habits

HABIT	THE RULE
The Docker Trigger	Once docker-compose up is running: no Instagram/Facebook until you shut the terminal.
The 10-Line Rule	On hard days — write just 10 lines of code. If you do that, the day is won.
The 3:1 Comment Rule	For every 1 line AI writes, you write 3 comments explaining why it works.
Daily Git Commit	At least one commit per work day. Your GitHub activity graph is your reputation.
Saturday Review	30 minutes every Saturday. Answer the review questions in writing.
Error-First Habit	Read the full traceback before Googling. Find the file and line number first.

WEEK 0 — Python Refresh + Django Hello World

Before Docker, before DRF — make sure your foundation is solid. Skip if you're confident.

WEEK 0	Python Refresh + Django Hello World

Mon	**1 hr 00 min** Python: decorators, list comprehensions, *args/**kwargs — read docs + write 5 practice exercises from scratch **30 min** Python: classes, inheritance, __init__ — write a Person → Student subclass **30 min** Git: create your learning repo, commit, push to GitHub. This is Day 1 of your public record.
Tue	**1 hr 00 min** Python: file I/O, exceptions, context managers — script that reads a CSV and handles bad rows gracefully **1 hr 00 min** Python: virtualenv, pip, requirements.txt — install Django 4.2, verify with `django-admin --version`
Wed	**1 hr 00 min** Django: run startproject. Study every file. Read settings.py top to bottom — understand each setting. **1 hr 00 min** Django: create a `polls` app, write one view, wire one URL — get 'Hello World' in browser
Thu	**1 hr 00 min** Django: create a Question model, run makemigrations + migrate, explore Django admin **30 min** Django shell: create/read/filter QuerySet objects interactively **30 min** SQL Day 1: SELECT, WHERE, ORDER BY — 10 queries directly in SQLite shell
Fri	**1 hr 00 min** SQL Day 2: JOIN, GROUP BY, COUNT — 10 more queries. You need this to understand the ORM. **1 hr 00 min** Django: Templates — render a list from DB to HTML. Context dict from view to template.
Sat	**30 min** SATURDAY REVIEW — answer questions below in writing **1 hr 30 min** Fix anything broken. Commit all code. Write a README for your repo.

DELIVERABLES

✓	Python script using a class, decorator, and file read — all in one file
✓	Django project with one URL that renders a template from a DB model
✓	GitHub repo with 5+ meaningful commits from this week
✓	20 raw SQL queries written and tested in SQLite shell

SATURDAY REVIEW (30 min)

1.	What does a Django migration actually do at the database level?

2.	What confused you most this week? Write 2 sentences.
3.	Did you commit every day? If not — what got in the way?

PHASE 1 — The Professional Foundation (Month 1, Weeks 1–4)

Docker, ORM, Performance, Middleware. You learn 'The Django Way.'

WEEK 1 — Docker + Professional Environment

Mon	**45 min** Install WSL2 + Docker Desktop. Verify: `docker run hello-world` works. **1 hr 15 min** Write a Dockerfile for a Python app. Build + run it. Read every line out loud.
Tue	**1 hr 00 min** Write docker-compose.yml with `web` (Django) and `db` (Postgres 15). No app yet — just get both containers running. **1 hr 00 min** Connect Django to Postgres via `DATABASE_URL`. Run `migrate` inside the container.
Wed	**1 hr 00 min** Add .env file. Use python-decouple to read env vars. Never hardcode passwords in code. **1 hr 00 min** Add volumes: Postgres data must survive `docker-compose down`. Test it: add data, restart, confirm it's still there.
Thu	**1 hr 00 min** Makefile with: `make up`, `make down`, `make shell`, `make migrate`. This is your workflow upgrade. **1 hr 00 min** Destroy everything: `docker-compose down -v`. Rebuild from scratch in under 5 minutes. Practice this.
Fri	**1 hr 00 min** Add pgAdmin container. Connect it to Postgres. Browse your Django tables visually. **1 hr 00 min** Write SETUP.md: step-by-step for a new developer to clone your repo and get it running. Follow your own instructions.
Sat	**30 min** SATURDAY REVIEW — answer questions below **1 hr 30 min** Clean docker-compose. Commit everything. Push.

DELIVERABLES

✓	Django running inside Docker, connected to Postgres
✓	.env file managing all secrets (zero hardcoded values)
✓	Postgres data persists across restarts (volumes confirmed working)
✓	Makefile with 5+ shortcuts
✓	SETUP.md that works when followed from scratch

SATURDAY REVIEW (30 min)

1.	What is the difference between a Docker image and a container?

2.	Why do we use volumes? What happens without them?
3.	What does `depends_on` do in docker-compose?

WEEK 2 — Advanced ORM + Custom User Model

Mon	**1 hr 00 min** Read Django docs: AbstractBaseUser vs AbstractUser. Understand the difference BEFORE writing any code. **1 hr 00 min** Create `users` app. Build CustomUser with email as login (no username). Run migrations.
Tue	**1 hr 00 min** Build CustomUserManager with create_user and create_superuser. Test in Django shell. **1 hr 00 min** Register in admin. Create superuser. Log in with email. Confirm it works.
Wed	**1 hr 00 min** Profile model: OneToOne to CustomUser (bio, profile_pic, location). Signal to auto-create Profile on User creation. **1 hr 00 min** ManyToMany: add `followers` field (self-referential). Write follow(user) and unfollow(user) methods.
Thu	**1 hr 00 min** Shell drill: create 5 users, A follows B and C. Query 'who does A follow', 'who follows B'. Verify results. **1 hr 00 min** ForeignKey: User has many Posts. Create 20 posts across 5 users in shell.
Fri	**1 hr 00 min** First test file: tests/test_models.py. Test user creation, follow/unfollow logic. Run with pytest. **1 hr 00 min** Admin: show follower count in User list. Add search and filter.
Sat	**30 min** SATURDAY REVIEW — answer questions below **1 hr 30 min** Refactor + clean up. Commit per feature with meaningful messages.

DELIVERABLES

✓	CustomUser with email-based login (no username field)
✓	Follow/unfollow via self-referential ManyToMany — working in shell
✓	Profile auto-created via Django signal
✓	First test file with passing model tests

SATURDAY REVIEW (30 min)

1.	Why must CustomUser be set up before the first migration?
2.	What is the difference between OneToOne and ForeignKey?

3.	What does a Django signal do and when would you use one?

WEEK 3 — ORM Performance — Solving the N+1 Problem

Mon	**1 hr 00 min** Read about N+1. Install Django Debug Toolbar in Docker. See query count on a view. **1 hr 00 min** Fetch 100 posts + authors WITHOUT optimization. Screenshot the query count.
Tue	**1 hr 00 min** Fix with select_related. See query count drop. Understand: select_related = SQL JOIN, one query. **1 hr 00 min** Fetch posts + tags (ManyToMany). See N+1. Fix with prefetch_related. Compare counts before and after.
Wed	**1 hr 00 min** only() and defer(): fetch 500 Users but only email + name. Measure query size vs full fetch. **1 hr 00 min** values() and values_list(): aggregate without model instantiation. Write 5 examples.
Thu	**1 hr 00 min** annotate() and aggregate(): count posts per user, get average follower count — in one query. **1 hr 00 min** Database indexes: add db_index=True to frequently filtered fields. Write a comment explaining why.
Fri	**1 hr 00 min** Build the script: 500 posts with authors, tags, comment count — in ≤ 3 SQL queries. Verify with Debug Toolbar. **1 hr 00 min** Write tests using assertNumQueries to lock in your query budget.
Sat	**30 min** SATURDAY REVIEW — answer questions below **1 hr 30 min** Document optimizations in README. Commit.

DELIVERABLES

✓	Django Debug Toolbar showing query counts per page
✓	Script fetching 500 records in ≤ 3 queries (verified, screenshotted)
✓	Tests using assertNumQueries to enforce performance budget
✓	Written explanation: when to use select_related vs prefetch_related

SATURDAY REVIEW (30 min)

1.	What is the N+1 problem? Give a real example from your code.
2.	When does select_related NOT work? What do you use instead?
3.	What does annotate() do that aggregate() doesn't?

WEEK 4 — Middleware + Forms + Request Logger

Mon	**1 hr 00 min** Read middleware docs. Trace the full request lifecycle: browser → middleware stack → view → back. **1 hr 00 min** Write TimingMiddleware: prints how long each request takes.
Tue	**1 hr 00 min** RequestLoggerMiddleware: saves IP, URL, method, status code, timestamp to DB on every request. **1 hr 00 min** Add exclusion logic: skip /admin/ and static files. Add management command to purge old logs.
Wed	**1 hr 00 min** Django Forms: UserRegistrationForm with custom validation (passwords match, email unique). **1 hr 00 min** Render in template. Handle POST. Show field errors inline. Redirect on success.
Thu	**1 hr 00 min** ModelForm: auto-generate from Profile model. Add clean_bio() with validation. **1 hr 00 min** File upload form: profile picture upload. Validate file type and max size.
Fri	**1 hr 00 min** Admin view for RequestLog: sortable, filterable list of all logged requests. **1 hr 00 min** Tests: middleware saves logs, form validation rejects bad input. 10+ passing tests total by now.
Sat	**30 min** SATURDAY REVIEW — Phase 1 retrospective: what do you know now vs 4 weeks ago? **1 hr 30 min** Write 5 bullet points about what you've learned. Commit everything.

DELIVERABLES

✓	RequestLogger saving every request to DB (IP, URL, method, status, time)
✓	Custom Django Form with validation rejecting bad input
✓	ModelForm with file upload, type and size validation
✓	10+ passing tests across all Phase 1 work

SATURDAY REVIEW (30 min)

1.	In what order do middlewares run? process_request vs process_response?
2.	What is the difference between a Form and a ModelForm?
3.	How does Django prevent CSRF attacks? What does the middleware do?

PHASE 2 — The API Specialist (Month 2, Weeks 5–8)

Stop building websites. Start building engines. Full DRF mastery.

WEEK 5 — HTTP Fundamentals + DRF Basics

Mon	**45 min** HTTP deep dive: GET/POST/PUT/PATCH/DELETE, status codes 200/201/400/401/403/404/500, headers. Write a cheatsheet. **1 hr 15 min** Install DRF. Build first APIView — GET endpoint returning a list of products as JSON.
Tue	**1 hr 00 min** Serializers: write plain Serializer for Product. Understand to_representation and validate_<field>. **1 hr 00 min** ModelSerializer: rewrite the same serializer. Compare code length vs plain Serializer.
Wed	**1 hr 00 min** Nested serializers: Category has many Products. Show category data inside each product response. **1 hr 00 min** POST endpoint: create a Product. Validate required fields. Return 201 on success, 400 on error.
Thu	**1 hr 00 min** PUT and PATCH: update a product. Understand full update vs partial. Test both in Postman or Thunder Client. **1 hr 00 min** DELETE endpoint. Return 204 No Content. Return 404 if product not found (get_object_or_404).
Fri	**1 hr 00 min** Install drf-spectacular. Auto-generate Swagger docs. Every endpoint visible at /api/docs/. **1 hr 00 min** API tests with APIClient: GET list, POST create, PUT update, DELETE. One test per endpoint minimum.
Sat	**30 min** SATURDAY REVIEW — answer questions below **1 hr 30 min** All endpoints return correct status codes. Clean up + commit.

DELIVERABLES

✓	Full CRUD JSON API for Products with nested Category data
✓	Swagger documentation accessible at /api/docs/
✓	API tests using DRF's APIClient (one per endpoint minimum)
✓	HTTP cheatsheet in your own words — keep this for interviews

SATURDAY REVIEW (30 min)

1.	What is the difference between PUT and PATCH?
2.	What HTTP status code should a successful POST return? Why not 200?

3.	What does a DRF Serializer actually do under the hood?

WEEK 6 — JWT Authentication + Custom Permissions

Mon	**1 hr 00 min** How JWT works: header.payload.signature. Decode a token at jwt.io. Access tokens vs refresh tokens. **1 hr 00 min** Install SimpleJWT. Add /api/token/ and /api/token/refresh/ endpoints. Test in Postman.
Tue	**1 hr 00 min** Protect an endpoint with IsAuthenticated. See 401 without token. See 200 with token. Test both. **1 hr 00 min** Custom Permission: IsOwnerOrReadOnly — only the creator can edit/delete their item.
Wed	**1 hr 00 min** Role-based permission: Admins can DELETE, regular users can only GET. Test as both user types. **1 hr 00 min** Throttling: 20 req/hour unauthenticated, 100 req/hour authenticated.
Thu	**1 hr 00 min** Registration endpoint: POST /api/register/ creates user and returns JWT tokens immediately. **1 hr 00 min** Password change endpoint: validate old password, accept new, handle token invalidation.
Fri	**1 hr 00 min** Security audit your own API: try to access another user's data. Fix any holes you find. **1 hr 00 min** Auth tests: register, login, access protected route, unauthorized delete — all passing.
Sat	**30 min** SATURDAY REVIEW — answer questions below **1 hr 30 min** Document all auth endpoints in Swagger with example request/response bodies.

DELIVERABLES

✓	JWT login flow: register → get token → access protected endpoint
✓	IsOwnerOrReadOnly permission (only owner modifies their content)
✓	Rate limiting active for both auth and unauth users
✓	Auth tests: register, login, forbidden access, token refresh — all passing

SATURDAY REVIEW (30 min)

1.	Why do we have both access and refresh tokens? Why not just one?
2.	What happens when an access token expires? Walk through the flow.
3.	What is the difference between Authentication and Authorization?

WEEK 7 — ViewSets + Routers + Task Manager API

Mon	**1 hr 00 min** Study GenericAPIView + Mixins: ListModelMixin, CreateModelMixin, etc. Rewrite Product API using them. **1 hr 00 min** ModelViewSet: convert to a single ViewSet. Observe how much code disappears.
Tue	**1 hr 00 min** DefaultRouter: register your ViewSet. See all URLs auto-generated. Print them with `show_urls`. **1 hr 00 min** Custom @action: add `POST /products/{id}/toggle_active/` action to the ViewSet.
Wed	**1 hr 00 min** Task Manager: Task model (title, description, status, due_date, owner ForeignKey to User). **1 hr 00 min** TaskViewSet: full CRUD. Only authenticated users create. Only owner updates/deletes.
Thu	**1 hr 00 min** Category model + GET /categories/{id}/tasks/ — lists tasks within a category. **1 hr 00 min** Override get_queryset() to only return logged-in user's tasks. No user sees another's tasks.
Fri	**1 hr 00 min** Custom serializer fields: `days_until_due` computed field, `is_overdue` boolean. **1 hr 00 min** Full test suite for Task Manager: CRUD, permission checks, ownership isolation. 15+ tests.
Sat	**30 min** SATURDAY REVIEW — answer questions below **1 hr 30 min** Add Swagger docs to Task Manager. Every endpoint, every field described.

DELIVERABLES

✓	Task Manager API via ModelViewSet + Router (full CRUD)
✓	Owner isolation: users only see/edit their own tasks
✓	Custom computed serializer fields (days_until_due, is_overdue)
✓	15+ tests covering CRUD, permissions, and data isolation

SATURDAY REVIEW (30 min)

1.	What does a Router actually do? What URLs does DefaultRouter create?
2.	When would you NOT use a ViewSet and use plain APIView instead?
3.	What is get_queryset() and why should you override it for security?

WEEK 8 — Filtering, Search + Pagination

Django Master Curriculum 2026	6 Months • 2 hrs/day • Saturday Reviews

Mon	**1 hr 00 min** Install django-filter. FilterSet for Task: filter by status, due_date range, category. **1 hr 00 min** Test in Postman: GET /tasks/?status=pending&category=1 — verify correct results.
Tue	**1 hr 00 min** DRF SearchFilter: full-text search on task title and description. Test edge cases. **1 hr 00 min** OrderingFilter: sort tasks by due_date, created_at, status. Test ascending and descending.
Wed	**1 hr 00 min** Pagination: PageNumberPagination (10 tasks/page). page and page_size on every list endpoint. **1 hr 00 min** CursorPagination: understand why it's better for large datasets and real-time feeds.
Thu	**1 hr 00 min** Combined: filter + search + ordering + pagination all on one endpoint. Test it thoroughly. **1 hr 00 min** Add DB indexes on fields used for filtering. Verify query count stays low with Debug Toolbar.
Fri	**1 hr 00 min** Phase 2 polish: remove print statements, fix TODOs, clean imports. **1 hr 00 min** Tests for filtering, search, pagination. At least 2 tests per feature.
Sat	**30 min** SATURDAY REVIEW — Phase 2 retrospective: you can now build a full REST API. What still feels shaky? **1 hr 30 min** List what's shaky. Plan how to address it. Commit Phase 2 final.

DELIVERABLES

✓	Search endpoint filtering by name, date range, and status simultaneously
✓	Pagination: correct page sizes, next/previous links in response
✓	Ordering: tasks sortable by multiple fields
✓	Tests for all filter/search/pagination combinations

SATURDAY REVIEW (30 min)

1.	What is the difference between filtering and searching in DRF?
2.	Why is cursor pagination better than page number for large tables?
3.	Which fields in your models should have database indexes and why?

PHASE 3 — Scaling & Infrastructure (Month 3, Weeks 9–12)

Redis, Celery, File Storage. The hard things AI struggles to debug for you.

WEEK 9 — Redis + Caching

Mon	**45 min** What is Redis? Key-value, in-memory, TTL. Add Redis to docker-compose. Verify with `redis-cli ping`. **1 hr 15 min** Configure Redis as Django cache backend. Write first cached view with @cache_page.
Tue	**1 hr 00 min** Low-level API: cache.get(), cache.set(), cache.delete(). Cache an expensive DB query manually. **1 hr 00 min** Measure: time an uncached vs cached endpoint. Document the numbers (e.g., 240ms → 12ms).
Wed	**1 hr 00 min** Cache invalidation: when a task updates, delete its cached response. Test stale data never shows. **1 hr 00 min** Per-user caching: cache key includes user ID. Test with 2 different users — no bleed-through.
Thu	**1 hr 00 min** Cache DRF views with cache_page + vary by user. Understand cache_control headers. **1 hr 00 min** Redis as session backend: move sessions from DB to Redis. Verify sessions survive requests.
Fri	**1 hr 00 min** redis-cli drill: monitor live cache hits/misses during requests. Observe TTL behavior. **1 hr 00 min** Tests: cached response < 5ms on second hit. Cache clears on model update.
Sat	**30 min** SATURDAY REVIEW — answer questions below **1 hr 30 min** Add Redis to Docker stack cleanly. Update SETUP.md with Redis instructions.

DELIVERABLES

✓	Redis in Docker, connected as Django cache backend
✓	Measurable speedup on cache hit (numbers documented in README)
✓	Cache invalidation working: updates clear stale cache
✓	Per-user cache keys (users never see each other's cached data)

SATURDAY REVIEW (30 min)

1.	What does TTL mean in Redis? What happens when a key expires?
2.	How do you decide what to cache and what not to?

3.	What is the hardest problem in caching? (Answer: invalidation — explain why.)

WEEK 10	Celery — Background Tasks

Mon	**1 hr 00 min** What is Celery? Task queue, workers, brokers. Add Celery + Redis broker to docker-compose. **1 hr 00 min** First task: @shared_task def add(x, y). Call with .delay(). See it run in worker logs.
Tue	**1 hr 00 min** Real task: send_welcome_email() triggered by user registration signal. Worker sends it in background. **1 hr 00 min** Simulate slow task: time.sleep(5) inside task. API returns instantly. Email arrives 5s later. This is the point.
Wed	**1 hr 00 min** Task retries: max_retries=3, retry_backoff=True. Test by raising exception inside the task. **1 hr 00 min** Task states: PENDING/STARTED/SUCCESS/FAILURE. Store task ID on model. Poll status via API endpoint.
Thu	**1 hr 00 min** Flower: add to docker-compose. Browse task history, retries, worker status visually. **1 hr 00 min** Chain tasks: task A result passed to task B. Example: scrape URL → summarize text.
Fri	**1 hr 00 min** CSV export: user requests export → Celery task generates CSV → model stores file path → user downloads. **1 hr 00 min** Test Celery tasks using task_always_eager=True in test settings. Write 5 task tests.
Sat	**30 min** SATURDAY REVIEW — answer questions below **1 hr 30 min** Celery + Redis stack clean. All services in docker-compose. Makefile updated.

DELIVERABLES

✓	Welcome email task running in Celery worker (not blocking API response)
✓	Retry logic with exponential backoff on failure
✓	Flower dashboard showing task history
✓	Task status polling endpoint (check if CSV export is ready)

SATURDAY REVIEW (30 min)

1.	What is the difference between .delay() and .apply_async()?
2.	Why should long-running tasks NEVER run inside a Django view?
3.	What happens to a task if the worker crashes mid-execution?

WEEK 11	File Storage — MinIO / S3

Mon	**45 min** Add MinIO (S3-compatible) to docker-compose. Access web console. Create a bucket. **1 hr 15 min** Install django-storages + boto3. Configure Django to use MinIO as DEFAULT_FILE_STORAGE.
Tue	**1 hr 00 min** ImageField on Profile. Upload picture — verify it lands in MinIO, NOT local disk. **1 hr 00 min** File validation: reject non-images and files > 2MB. Test with intentionally bad uploads.
Wed	**1 hr 00 min** Pre-signed URLs: time-limited URL for private files. URL expires after 1 hour. **1 hr 00 min** Celery task: after upload, resize to 200x200 thumbnail with Pillow. Store separately.
Thu	**1 hr 00 min** PDF upload: store in MinIO. Serve via pre-signed URL. Never expose the bucket directly. **1 hr 00 min** Cleanup task: Celery Beat job that deletes orphaned MinIO files nightly.
Fri	**1 hr 00 min** Test uploads via APIClient: upload image, verify URL accessible, verify MinIO has the file. **1 hr 00 min** Document file upload API in Swagger: accepted types, max size, example response.
Sat	**30 min** SATURDAY REVIEW — answer questions below **1 hr 30 min** Update SETUP.md with MinIO setup. Fresh clone → full stack → file upload. Confirm it works.

DELIVERABLES

✓	Profile picture upload storing to MinIO (not local disk)
✓	Thumbnail generation via Celery background task
✓	Pre-signed URLs with expiry for private file access
✓	File validation rejecting wrong types and oversized files

SATURDAY REVIEW (30 min)	
1.	Why should you never store uploaded files on the same server as your app?
2.	What is a pre-signed URL and when would you use one?
3.	How do you prevent a malicious file disguised as an image?

WEEK 12	Celery Beat — Scheduled Tasks

Django Master Curriculum 2026	6 Months • 2 hrs/day • Saturday Reviews

Mon	**1 hr 00 min** Add celery-beat to docker-compose. Difference: Celery = tasks, Beat = scheduler. **1 hr 00 min** First beat schedule: health_check task every minute. Verify in Flower it fires on time.
Tue	**1 hr 00 min** Delete expired tasks: status=expired AND due_date < today. Schedule at midnight. **1 hr 00 min** Daily summary email: tasks due tomorrow, per user. Schedule at 9 AM daily.
Wed	**1 hr 00 min** django-celery-beat: manage schedules from Django admin (no code changes needed). **1 hr 00 min** Task locking: prevent same task running twice if previous is still running.
Thu	**1 hr 00 min** Phase 3 integration: endpoint → Celery task → Redis cache → MinIO file → all in one flow. **1 hr 00 min** Monitor full stack: Django, Postgres, Redis, Celery, Beat, MinIO — all 6 running, no errors.
Fri	**1 hr 00 min** Phase 3 code review: read all your own code from Weeks 9-12. List what you'd improve. **1 hr 00 min** Full test suite: all tests passing with pytest. Fix any broken ones.
Sat	**30 min** SATURDAY REVIEW — Phase 3 retrospective **1 hr 30 min** Full stack running. SETUP.md updated. GitHub pushed. You now have a real backend stack.

DELIVERABLES

✓	Midnight cleanup task: deletes expired tasks automatically
✓	Daily email summary task scheduled and confirmed in Flower
✓	All 6 Docker services running cleanly with no errors in logs
✓	Full test suite passing after all of Phase 3

SATURDAY REVIEW (30 min)

1.	What is the difference between Celery and Celery Beat?
2.	What happens if a scheduled task takes longer than its interval?
3.	Looking at your Phase 3 code — what would you refactor first?

PHASE 4 — The Job-Winner Projects (Months 4–6)

Build real things. Deploy them. Get hired.

WEEK 13–16 | Month 4 — Project 1: The AI Engine

Week 13	**Mon–Fri, 2 hrs/day** Plan + scaffold: Django project, Postgres, Docker. Define ScrapeJob model (url, status, result_text, summary, created_at).
Week 14	**Mon–Fri, 2 hrs/day** Build the scraper: requests + BeautifulSoup (or Scrapy for JS sites). Celery task scrapes URL, stores raw text in ScrapeJob.
Week 15	**Mon–Fri, 2 hrs/day** LLM integration: send scraped text to OpenAI API (or Ollama locally). Store AI summary. Handle token limits by chunking long text.
Week 16	**Mon–Fri, 2 hrs/day** Polish: full DRF API, JWT auth, rate limiting, Swagger docs, 80% test coverage, README with architecture diagram.

DELIVERABLES

✓	POST /api/scrape/ accepts URL, returns job ID immediately (non-blocking)
✓	GET /api/scrape/{id}/ returns status + AI summary when complete
✓	LLM integration working (OpenAI or local Ollama)
✓	Full Swagger documentation + 80% test coverage
✓	GitHub repo with clean README explaining the full system

SATURDAY REVIEW (30 min)

1.	What was the hardest technical problem you solved this month?
2.	How did you handle URLs with content too long for the LLM context window?
3.	Could this API be abused? What rate limiting did you add?

WEEK 17–20 | Month 5 — Project 2: Real-Time Notifications

Week 17	**Mon–Fri, 2 hrs/day** Django Channels setup: add Channels + Daphne to Docker. Replace Django dev server with ASGI.

Week 18	**Mon–Fri, 2 hrs/day** WebSocket consumer: user connects → authenticated → joins personal channel group. Send test message from server.
Week 19	**Mon–Fri, 2 hrs/day** Notification system: when a task is assigned, Celery task fires a WebSocket push to that user's channel group.
Week 20	**Mon–Fri, 2 hrs/day** Polish: notification history API, mark-as-read endpoint, reconnection logic, load test 50 concurrent WebSocket connections.

DELIVERABLES

✓	WebSocket connection with JWT auth (unauthenticated connections rejected)
✓	Real-time push on task assignment (browser receives within 1 second)
✓	Notification history stored in DB + retrieval API
✓	Mark-as-read endpoint
✓	Load test: 50 concurrent WebSocket connections handled without crash

SATURDAY REVIEW (30 min)

1.	What is the difference between HTTP and WebSocket? When do you use each?
2.	How does Django Channels use Redis differently from Celery?
3.	What happens when a user is offline — how do you store missed notifications?

WEEK 21–24 Month 6 — Testing, DevOps + Job Prep

Week 21	**Mon–Fri, 2 hrs/day** Pytest mastery: fixtures, parametrize, mocking with unittest.mock. Bring both projects to 80%+ coverage with pytest-cov.
Week 22	**Mon–Fri, 2 hrs/day** GitHub Actions: CI pipeline runs tests on every push. Failed tests block the push. Coverage badge in README.
Week 23	**Mon–Fri, 2 hrs/day** Deploy AI Engine to Render (free tier). Production settings: DEBUG=False, ALLOWED_HOSTS, secure SECRET_KEY, Postgres addon.
Week 24	**Mon–Fri, 2 hrs/day** Job prep: write resume, polish GitHub profile, prepare 10 Django interview Q&A, do 2 mock interviews (ask Claude).

DELIVERABLES

✓	Both projects at 80%+ coverage (verified with pytest-cov)

✓	GitHub Actions CI running and blocking bad pushes
✓	AI Engine deployed and publicly accessible on Render
✓	GitHub profile: 2 pinned repos with READMEs and architecture diagrams
✓	Resume with backend engineer positioning + 10 interview answers written

SATURDAY REVIEW (30 min)	
1.	What is a pytest fixture and why is it better than setUp()?
2.	What does a CI/CD pipeline do? Explain it to a non-developer.
3.	What are the 3 most important things you learned in 6 months?

By Month 6, You Will Have:

→ 2 deployed, public projects with Swagger docs and CI/CD pipelines
→ A GitHub profile showing consistent daily commits for 6 months
→ Real experience with Docker, Celery, Redis, WebSockets, and AI APIs
→ **A resume that says Backend Engineer — and actually means it.**